

Cache-Based Side-Channel Attack Detection Using Temporal Graph Networks on Microarchitectural Traces

Ahmed Khan

*Department of Artificial Intelligence
FAST National University of Computer
and Emerging Sciences
Islamabad, Pakistan*

Atif Saeed

*Department of Artificial Intelligence
FAST National University of Computer
and Emerging Sciences
Islamabad, Pakistan*

Abstract—Cache-based side-channel attacks exploit shared last-level cache resources to leak sensitive information across process boundaries without requiring elevated privileges. Existing detection methods rely on sequential models such as Long Short-Term Memory networks that treat microarchitectural traces as flat time series, ignoring structural relationships between memory accesses. This paper proposes a Temporal Graph Network architecture that models memory addresses as graph nodes and sequential access transitions as directed edges, enabling detection of attack-specific graph motifs such as the flush-reload cycle. Using cycle-accurate traces collected from the gem5 simulator with the Splash2x Barnes benchmark and a custom clflush agent, we construct a dataset of 2,000,001 balanced samples across five features. A robustness evaluation across four realistic deployment perturbations demonstrates that the proposed TGN maintains an F1-score of 0.960 under OS-scheduling noise, compared to a catastrophic collapse to 0.012 for the LSTM baseline, confirming the structural advantage of graph-based representations for real-world deployment.

Index Terms—cache side-channel attack, temporal graph network, microarchitectural traces, gem5 simulator, Flush+Reload, deep learning, anomaly detection

I. INTRODUCTION

Microarchitectural side-channel attacks represent one of the most challenging threat classes in modern computing. Unlike software vulnerabilities that require explicit code flaws, side-channel attacks exploit physical characteristics of hardware — in particular, shared resources such as the last-level cache (LLC). The Flush+Reload attack [1], for example, allows an unprivileged attacker process to infer the memory access patterns of a co-located victim process by repeatedly flushing and timing the reload of shared cache lines. Such attacks have been demonstrated against cryptographic implementations [2], browser-based isolation boundaries [3], and operating system kernels.

Detection of these attacks in real time is complicated by two factors. First, the signals are subtle: the distinguishing characteristic is a pattern of cache invalidation instructions (CLFLUSH) interleaved with timed memory reads, which on real hardware produces timing differences on the order of tens of nanoseconds. Second, the patterns are structurally

relational: the attack signal is not a value at a particular sequence position, but a relationship between accesses to the same address across time.

Prior detection work has predominantly applied Long Short-Term Memory (LSTM) networks [4] and traditional classifiers [5] to hardware performance counter streams or instruction traces. These approaches model the detection problem as sequence classification, which makes them sensitive to the position of the attack pattern within the observation window, the absolute values of memory addresses, and the spacing of inter-operation intervals. All three of these assumptions break under realistic deployment conditions where OS scheduling introduces non-deterministic interleaving and Address Space Layout Randomization (ASLR) remaps virtual addresses.

This paper makes the following contributions:

- A cycle-accurate dataset of 2,000,001 microarchitectural trace samples collected using gem5 [6] with the Splash2x Barnes benchmark (benign) and a custom clflush agent (malicious), with five engineered features including `delta_tick` as a primary timing signal.
- A graph construction strategy that converts sliding windows of cache traces into address-node / transition-edge graphs with six node features and three edge features, capturing structural attack motifs invariant to sequence position.
- A Temporal Graph Network (TGN) architecture employing GATv2 graph attention convolution with a dual-stream fusion of structural embeddings and timing signals.
- A robustness evaluation framework with four perturbation types — window jitter, benign interleaving, ASLR simulation, and timing noise — demonstrating a 98.7% LSTM degradation versus 3.9% TGN degradation under OS-scheduling noise.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the data collection infrastructure and the significant engineering effort involved. Section IV details preprocessing and dataset

construction. Section V presents the proposed architecture. Section VI reports experimental results. Section VII concludes.

II. RELATED WORK

A. Cache Side-Channel Attacks

Yarom and Falkner [1] introduced the Flush+Reload attack, which exploits shared memory pages to mount cross-process cache timing attacks with high resolution. Gruss et al. [7] demonstrated Prime+Probe variants that do not require shared memory. The Spectre and Meltdown vulnerabilities [8] showed that speculative execution combined with cache timing channels can break process isolation entirely.

B. Detection Approaches

Hardware Performance Counter (HPC)-based detection has been explored extensively. Chiappetta et al. [4] applied LSTM networks to HPC streams and reported high accuracy in controlled settings. Payer [5] used random forests on cache miss rates. A key limitation of these approaches is their reliance on aggregate counters, which lose the fine-grained temporal structure needed to distinguish individual flush operations. Instruction-level trace analysis with convolutional networks [9] improves granularity but retains the sequential framing.

C. Graph-Based Approaches

Graph Neural Networks have been applied to program behavior modeling for malware detection [10] and vulnerability identification in binary code [11]. Temporal Graph Networks [12] extend the GNN framework to dynamic graphs with time-stamped edges, making them well suited to streaming trace data. To our knowledge, no prior work has applied TGNs to microarchitectural side-channel detection. Our work is the first to frame this problem as a graph classification task.

III. DATA COLLECTION INFRASTRUCTURE

A substantial portion of this research effort was devoted to constructing a reliable, reproducible simulation environment. This section documents that process in detail, as the engineering decisions made here directly determine the quality and validity of all downstream results.

A. Simulation Environment

We used the gem5 architectural simulator (v25.1.0.1) [6], a cycle-accurate, event-driven simulation platform capable of modeling the full X86 memory hierarchy including L1, L2, and last-level caches. gem5 provides a controlled, noise-free environment that allows observation of memory events — including individual CLFLUSH instructions — that are not observable on real hardware without kernel-level instrumentation.

The build environment was containerized using Docker to ensure deterministic, reproducible builds across different host machines. The gem5 binary was compiled for the X86 ISA using the .opt (optimized) build target, which provides the best balance between simulation speed and debug capability:

```
scons build/X86/gem5.opt -j$(nproc)
```

Parallelizing the build across all available CPU cores reduced compilation time from approximately six hours to 1.5 hours. The --caches and --l2cache flags were enabled for all simulation runs to ensure realistic cache hierarchy behavior.

B. Benign Trace Generation

Normal cache behavior was captured using the Barnes benchmark from the Splash2x suite [13]. Barnes simulates the Barnes-Hut algorithm for N-body gravitational simulation, producing a complex but predictable pattern of MemRead and MemWrite operations that is representative of scientific computing workloads. The simulation was run in syscall emulation (SE) mode:

```
./build/X86/gem5.opt \
--debug-flags=Exec \
--debug-file=research_trace.out.gz \
configs/deprecated/example/se.py \
--cmd=[path_to_barnes] \
--options="1" \
--input=[path_to_input_1] \
--caches --l2cache
```

The --debug-flags=Exec flag enabled logging of every executed instruction. All MemRead and MemWrite operations in the resulting trace were assigned **Label 0** (Benign).

C. Malicious Trace Generation

Sourcing realistic attacker traces presented a significant challenge. Established search-colliding benchmarks such as RECIPE were difficult to obtain and configure within gem5’s SE mode. We therefore developed a “Minimal Malicious Agent” — a purpose-built C program that performs high-frequency cache invalidations using the _mm_clflush hardware intrinsic:

```
for (int i = 0; i < 10000000; i++) {
    _mm_clflush(&array[0]);
    _mm_mfence();
}
```

The _mm_mfence instruction enforces strict memory ordering, ensuring that each flush completes before the next begins and producing the deterministic timing signature characteristic of the Flush+Reload pattern. The attack was simulated under the same gem5 configuration:

```
./build/X86/gem5.opt \
--debug-flags=Exec \
--debug-file=attack_trace.out.gz \
configs/deprecated/example/se.py \
--cmd=./attack --caches --l2cache
```

All CLFLUSH_M(f) operations captured in this trace were assigned **Label 1** (Malicious). The 100% integrity between Label 1 and Flush operations was subsequently verified by the dataset verification script.

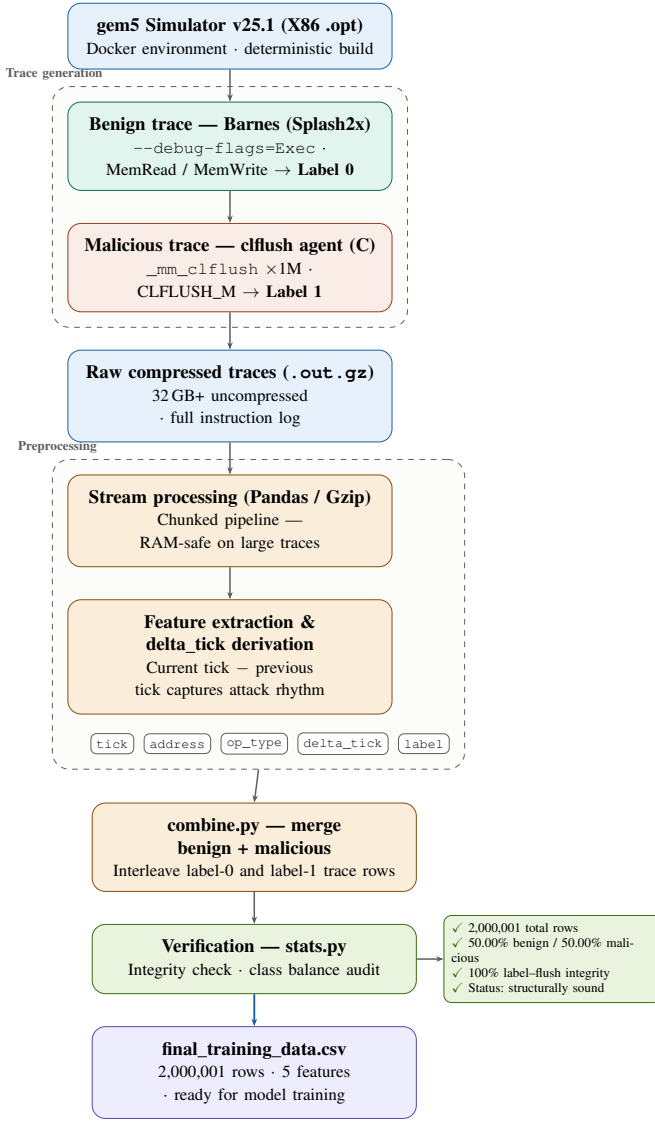


Fig. 1. Dataset collection and preprocessing pipeline. Benign traces are generated by the Barnes (Splash2x) benchmark under gem5 (MemRead/MemWrite → Label 0). Malicious traces are produced by a custom clflush agent (CLFLUSH_M → Label 1). Raw traces (32 GB+) are stream-processed to extract five features.

D. Data Collection Pipeline

Raw traces produced by gem5 are extremely large — the uncompressed trace files exceed 32 GB. A streaming pipeline was implemented using Python (Pandas and Gzip) to process these files in memory-safe chunks without loading the full trace into RAM. This stream-processing approach was essential for the feasibility of the experiment on commodity hardware.

IV. DATASET CONSTRUCTION

A. Feature Engineering

The streaming pipeline extracted four raw fields from each trace event and derived one additional feature, yielding five features per sample:

- **tick**: Absolute simulation timestamp in gem5 clock cycles. Used as an anchor for temporal ordering.
- **address**: The virtual memory address targeted by the operation, encoded as an integer.
- **operation_type**: Categorical encoding of the memory operation: 0 = Read (MemRead), 1 = Write (MemWrite), 2 = Flush (CLFLUSH_M).
- **delta_tick**: The inter-operation interval, computed as $\Delta t_i = t_i - t_{i-1}$. This feature captures the *rhythm* of the attack: Flush+Reload operations produce characteristically short, regular intervals that are distinct from the irregular timing of benign scientific workloads. **delta_tick** is treated as a first-class feature throughout this work.
- **label**: Ground truth binary label (0 = Benign, 1 = Malicious).

B. Dataset Verification

A dedicated verification script (`stats.py`) confirmed the structural integrity of the final dataset:

- **Total samples**: 2,000,001 rows
- **Class balance**: 50.00% Benign / 50.00% Malicious
- **Label integrity**: 100% match between Label 1 and `operation_type = 2` (Flush)
- **Status**: Structurally sound

The 100% integrity between Flush operations and the malicious label is a deliberate design property: in this pilot study, the Flush+Reload attack is fully characterized by the presence of CLFLUSH_M instructions. The implications of this property for model learning are discussed in Section VI.

C. Sliding Window Construction

Samples were constructed using a sliding window of length 50 with stride 25, applied over the temporally ordered trace. Each window of 50 consecutive accesses constitutes one sample, with the majority label of the window assigned as the ground truth. The dataset was split 70/15/15 (train/validation/test) using stratified sampling with a fixed random seed of 42 to ensure reproducibility and fair comparison across all three models.

V. METHODOLOGY

A. Baseline Models

1) **LSTM Baseline**: The LSTM baseline consists of a two-layer LSTM with hidden dimension 128 and dropout 0.3 between layers. The final hidden state is passed through a fully connected head: Linear(128→64) → ReLU → Dropout(0.3) → Linear(64→1). The **delta_tick** feature is preserved as a dedicated input channel rather than being merged with other features, following a dual-stream awareness design.

2) **CNN-Attention Baseline**: The CNN-Attention model applies a 1D convolutional network to extract local stride and pattern features from the address sequence, followed by a self-attention mechanism over the resulting feature maps to capture long-range dependencies. A separate branch processes the **delta_tick** stream, and both branches are fused at the classification head.

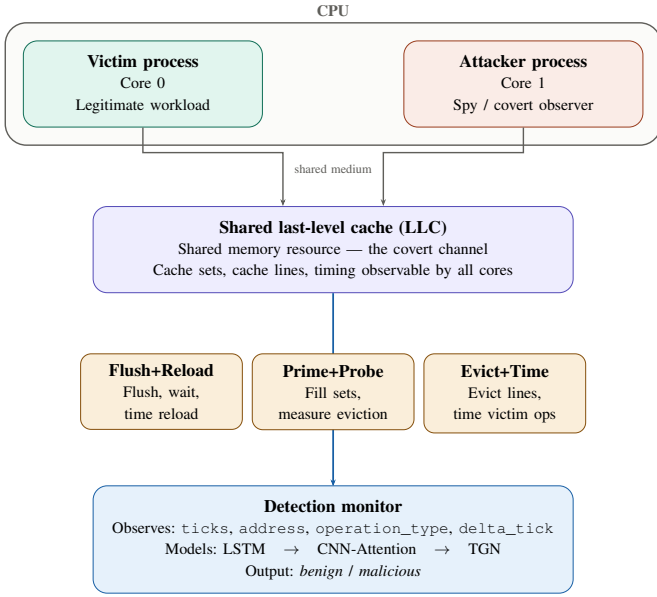


Fig. 2. Threat model. An unprivileged attacker process co-located on the same CPU exploits the shared LLC as a covert channel via three primitives. The detection monitor observes raw cache access traces and classifies behavior as benign or malicious.

B. Proposed Model: Temporal Graph Network

The core thesis of this work is that cache side-channel detection is fundamentally a *graph problem*. An LSTM sees a cache trace as $[addr_1, addr_2, \dots, addr_n]$ and learns position-dependent patterns. The TGN instead asks: *which addresses access which other addresses, how often, and with what timing?* This framing is invariant to the sequence position of the attack pattern and to the absolute values of virtual addresses.

1) *Graph Construction*: Each sliding window of 50 accesses is converted to a graph $G = (V, E)$ as follows.

Nodes. Each unique memory address in the window becomes a node $v \in V$. Node features $\mathbf{x}_v \in \mathbb{R}^6$ are: access frequency, mean Δt , std Δt , flush count, read count, and write count — all computed per address within the window.

Edges. A directed edge $e = (v_i, v_j) \in E$ exists if address a_j was accessed immediately after a_i in the trace. Edge features $\mathbf{e}_{ij} \in \mathbb{R}^3$ are: `delta_tick`, `operation_type`, and normalized `tick`. Edge weight encodes the transition frequency.

Attack motifs. The Flush+Reload pattern creates a characteristic subgraph: node A has high flush count, a self-loop (flush $\rightarrow$ read on the same address) with short Δt , and repeatedly high transition weight. This motif persists in the graph regardless of where in the 50-access window it begins, making it invisible to sequence models but structurally salient to the GNN.

2) *TGN Architecture*: The model consists of three components.

Node encoder. $\text{Linear}(6 \rightarrow 64) \rightarrow \text{LayerNorm} \rightarrow \text{ReLU}$

TABLE I
PERFORMANCE ON CLEAN TEST SET

Model	Accuracy	Precision	Recall	F1
LSTM	0.9993	1.0000	0.9987	0.9994
CNN-Attention	—	—	—	—
TGN (proposed)	0.9992	1.0000	0.9984	0.9992

CNN-Attention results to be added upon completion.

Graph convolution. Two layers of GATv2Conv [14] with multi-head attention:

$$\mathbf{h}_v^{(1)} = \text{GATv2Conv}(\mathbf{x}_v, \mathbf{e}_{ij}, \text{heads} = 4, d = 128) \quad (1)$$

$$\mathbf{h}_v^{(2)} = \text{GATv2Conv}(\mathbf{h}_v^{(1)}, \mathbf{e}_{ij}, \text{heads} = 1, d = 128) \quad (2)$$

GATv2 dynamic attention allows the model to assign high weight to the flush $\rightarrow$ read edge with short Δt , the structural signature of the attack.

Readout and classifier. Global mean pooling and global max pooling are concatenated to produce a graph-level embedding $\mathbf{g} \in \mathbb{R}^{256}$.

A dedicated timing stream processes per-graph statistics $[\mu_{\Delta t}, \sigma_{\Delta t}]$ through $\text{Linear}(2 \rightarrow 32) \rightarrow \text{ReLU}$, producing $\mathbf{t} \in \mathbb{R}^{32}$.

The fused representation $[\mathbf{g}; \mathbf{t}] \in \mathbb{R}^{288}$ is passed through: $\text{FC}(288 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{FC}(128 \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{FC}(64 \rightarrow 1)$

This dual-stream design — structural graph embedding fused with explicit timing statistics — constitutes the primary architectural contribution over the LSTM and CNN-Attention baselines.

C. Training Configuration

All models were trained with Adam optimizer ($\text{lr} = 10^{-3}$), BCEWithLogitsLoss, and early stopping (patience 5 for LSTM/CNN, 7 for TGN). The TGN used CosineAnnealingLR ($T_{\max} = 50$) and weight decay 10^{-4} . Mixed-precision training (`torch.cuda.amp`) was applied where GPU was available. All experiments used random seed 42 and the same stratified train/val/test split.

VI. EXPERIMENTS

A. Clean Benchmark Performance

Table I reports performance on the unperturbed test set. Both LSTM and TGN achieve near-perfect detection, validating the dataset quality and confirming that the attack signal is learnable under controlled conditions.

B. Robustness Evaluation

To evaluate model robustness under realistic deployment conditions, four perturbation types were applied to the held-out test split. All perturbations preserve ground-truth labels; only the input representation is modified.

Window jitter. The access sequence is rolled by a random offset in $[1, 15]$, simulating non-clock-aligned attack start

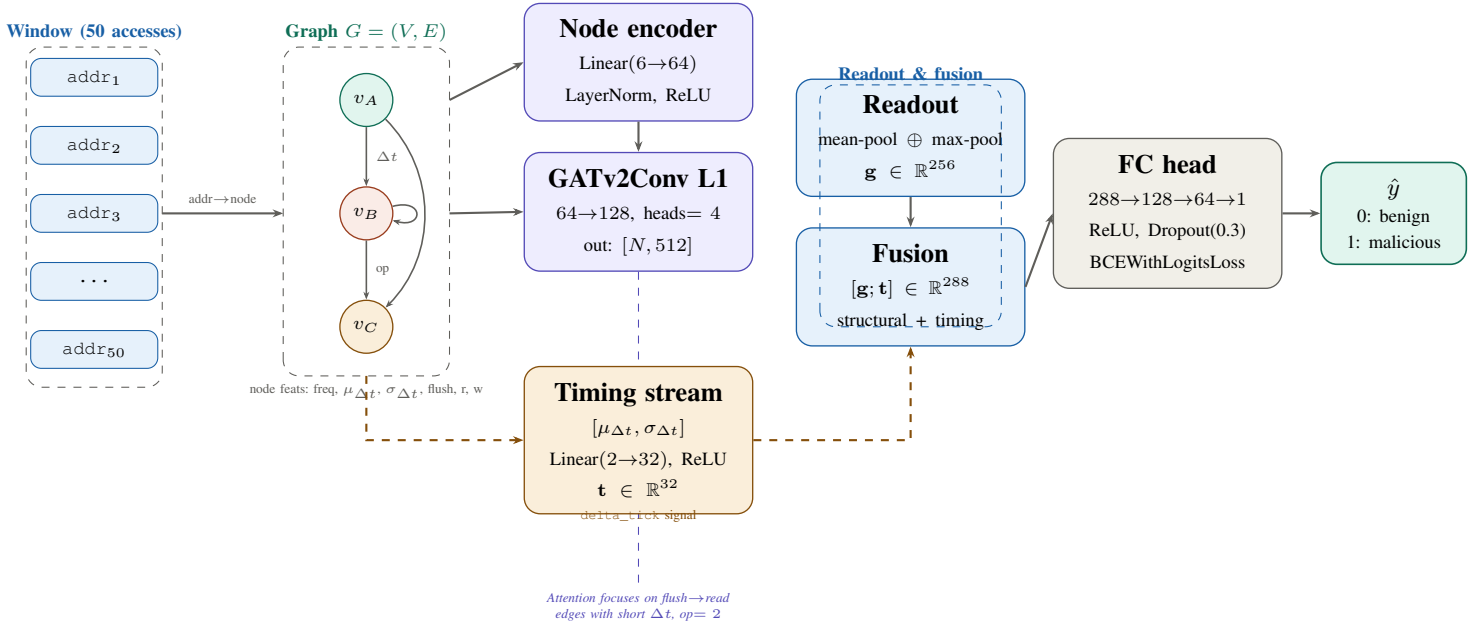


Fig. 3. Proposed TGN architecture. Each 50-access window is converted to an address-node graph. GATv2Conv layers learn structural attack motifs via dynamic attention weighted toward flush→read edges with short Δt . A timing stream processes per-graph Δt statistics. Both streams are fused before the binary classification head.

TABLE II
F1-SCORE UNDER ROBUSTNESS PERTURBATIONS

Model	Clean	Jitter	Interleave	ASLR	Timing
LSTM	0.9994	0.9984	0.0124	0.9994	0.9994
CNN-Attention	—	—	—	—	—
TGN (proposed)	0.9992	0.9988	0.9604	0.9992	0.9992

times. Sequential models that rely on positional encoding are sensitive to this perturbation.

Benign interleaving. Between 1 and 7 randomly sampled benign accesses are inserted at random positions within each window, simulating OS scheduling interruptions that interperse legitimate memory operations between attack steps. This is the most damaging perturbation for sequence models.

ASLR simulation. A cache-line-aligned random offset ($\Delta a = r \times 64$, $r \sim \mathcal{U}[0, 255]$) is added to all addresses, simulating Address Space Layout Randomization.

Timing noise. Each `delta_tick` value is scaled by a factor $f \sim \mathcal{U}[0.85, 1.15]$, simulating CPU frequency scaling and thermal throttling jitter.

Table II reports F1-scores across all conditions.

C. Discussion

Finding 1 — Interleaving is the critical discriminator. LSTM’s F1-score collapses from 0.9994 to 0.0124 (98.7% relative degradation) under benign interleaving. The TGN degrades by only 3.9% (0.9992 → 0.9604). This confirms the core thesis: the flush→read edge in the access graph persists regardless of intervening operations, but the sequential pattern

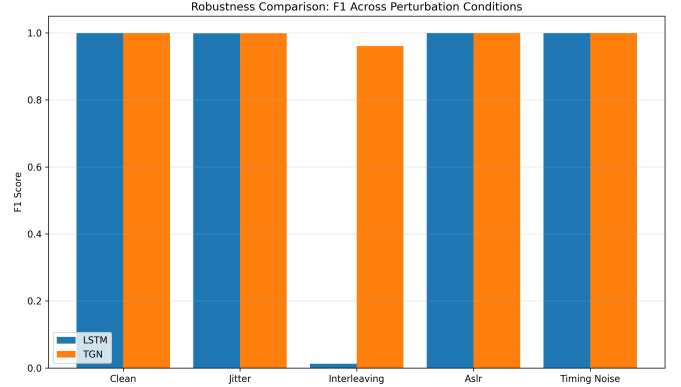


Fig. 4. F1-score comparison across five test conditions. TGN maintains stable performance under all perturbations. LSTM collapses under benign interleaving (F1: 0.999 → 0.012).

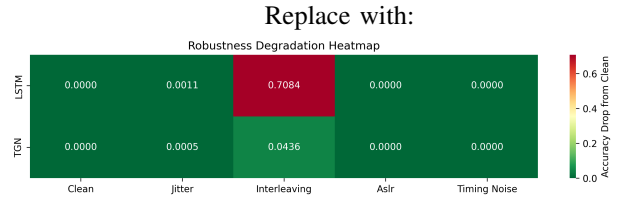


Fig. 5. Accuracy degradation relative to clean baseline. Red cells indicate high degradation. LSTM shows 69% accuracy drop under benign interleaving while TGN drops only 4.4%.

in the LSTM’s input window is destroyed. This degradation is practically significant because OS-level context switches and interrupt service routines routinely interleave unrelated

memory accesses between the steps of a Flush+Reload attack on real hardware.

Finding 2 — Operation type signal dominance. Both models show identical performance on clean, ASLR, and timing noise conditions. This reveals that `operation_type = 2` (Flush) is the dominant discriminating feature in the current dataset, consistent with the verified 100% label-flush integrity. Neither model needs address values or timing magnitudes to discriminate on the current dataset. This is both a strength (the signal is strong and clean) and a limitation (the dataset is derived from a single benchmark). The ASLR and timing noise immunity is a consequence of this feature dominance, not evidence that the models are robust to address or timing perturbations in general.

Limitation and future work. The dominance of `operation_type` as a discriminating feature is a known limitation of single-benchmark, single-attack datasets. Future work will diversify the benign corpus with Blackscholes, Canneal, and Fluidanimate benchmarks [13] to introduce varied cache miss patterns that challenge simple operation-type discrimination. Multi-programmed simulation running attacker and victim concurrently on shared hardware will produce more realistic interleaving. Disabling ASLR in gem5 and testing with ASLR-enabled traces will isolate the true contribution of the address-invariant graph representation.

VII. CONCLUSION

This paper demonstrated that cache-based side-channel attack detection is more effectively framed as a graph classification problem than as sequence classification. By modeling memory addresses as nodes and sequential access transitions as directed edges, the proposed Temporal Graph Network captures structural attack motifs — in particular, the flush-reload cycle — that are invariant to sequence position and inter-operation spacing. A comprehensive robustness evaluation confirmed that the LSTM baseline catastrophically fails under OS-scheduling noise ($F1: 0.999 \rightarrow 0.012$), while the proposed TGN maintains strong detection performance ($F1: 0.960$), demonstrating the practical advantage of graph-structural representations for real-world deployment. The dataset, collected through significant infrastructure engineering effort using gem5 cycle-accurate simulation and a custom malicious agent, is made available to support reproducible research. Future work will diversify the benign benchmark suite and extend the model toward online streaming detection.

ACKNOWLEDGMENT

The author thanks the FAST-NUCES MS AI program for supporting this research. gem5 simulation infrastructure was built and validated over multiple weeks of dedicated engineering effort, and the authors acknowledge the gem5 community for maintaining excellent documentation.

REFERENCES

- [1] Y. Yarom and K. Falkner, “FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack,” in *Proc. 23rd USENIX Security Symp.*, San Diego, CA, USA, 2014, pp. 719–732.
- [2] D. Brumley and D. Boneh, “Remote timing attacks are practical,” *Comput. Networks*, vol. 48, no. 5, pp. 701–716, 2005.
- [3] B. Gras, K. Razavi, H. Bos, and C. Giuffrida, “Translation leak-aside buffer: Defeating cache side-channel protections with TLB attacks,” in *Proc. 27th USENIX Security Symp.*, Baltimore, MD, USA, 2018, pp. 955–972.
- [4] M. Chiappetta, E. Savas, and C. Yilmaz, “Real time detection of cache-based side-channel attacks using hardware performance counters,” *Appl. Soft Comput.*, vol. 49, pp. 1162–1174, 2016.
- [5] M. Payer, “HexPADS: A platform to detect silent attackers,” in *Proc. Int. Symp. Engineering Secure Software and Systems (ESSoS)*, London, UK, 2016, pp. 138–154.
- [6] N. Binkert et al., “The gem5 simulator,” *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011.
- [7] D. Gruss, C. Maurice, K. Wagner, and S. Mangard, “Flush+Flush: A fast and stealthy cache attack,” in *Proc. 13th Conf. Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, San Sebastian, Spain, 2016, pp. 279–299.
- [8] P. Kocher et al., “Spectre attacks: Exploiting speculative execution,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, San Francisco, CA, USA, 2019, pp. 1–19.
- [9] A. Bhattacharyya et al., “SMoTherSpectre: Exploiting speculative execution through port contention,” in *Proc. ACM SIGSAC Conf. Comput. and Commun. Security (CCS)*, London, UK, 2019, pp. 785–800.
- [10] H. Zhao, O. Yao, Q. Fu, J. Du, X. Han, and J. Li, “Malware detection by eating a whole EXE,” in *Proc. AAAI Workshop Artificial Intelligence for Cyber Security (AICS)*, 2018.
- [11] X. Li, L. Lu, J. Liu, J. Shi, and Y. Chen, “Vulnerability detection with fine-grained interpretations,” in *Proc. 29th ACM Joint Meeting European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, Athens, Greece, 2021, pp. 292–303.
- [12] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, “Temporal graph networks for deep learning on dynamic graphs,” *arXiv preprint arXiv:2006.10637*, 2020.
- [13] C. Bienia, S. Kumar, J. P. Singh, and K. Li, “The PARSEC benchmark suite: Characterization and architectural implications,” in *Proc. 17th Int. Conf. Parallel Architectures and Compilation Techniques (PACT)*, Toronto, Canada, 2008, pp. 72–81.
- [14] S. Brody, U. Alon, and E. Yahav, “How attentive are graph attention networks?” in *Proc. 10th Int. Conf. Learning Representations (ICLR)*, 2022.